Unit 3: Strings and Arrays (8 Hrs.)

Covers:

- String Constants
- Printing Strings
- Accessing Characters in Strings
- Cleaning Strings
- Encoding and Escaping
- Comparing, Manipulating and Searching Strings
- Regular Expressions
- Array
- Indexed vs. Associative Arrays
- Defining Array
- Storing Data in Array
- Multidimensional Array
- Extracting Multiple Values
- Conversion between Array and Variables
- Traversing Arrays

String Constants:

PHP String

PHP string is a sequence of characters i.e., used to store and manipulate text. PHP supports only 256-character set and so that it does not offer native Unicode support.

Quoting String Constants

There are four ways to write a string literal in your PHP code: using single quotes, double quotes, the here document (heredoc) format derived from the Unix shell, and its "cousin" now document (nowdoc). These methods differ in whether they recognize special escape sequences that let you encode other characters or interpolate variables.

Variable Interpolation

When you define a string literal using double quotes or a heredoc, the string is sub ject to variable interpolation. Interpolation is the process of replacing variable names in the string with their contained values.

There are two ways to interpolate variables into strings.

The simpler of the two ways is to put the variable name in a double-quoted string or in a heredoc:

```
$who = 'Kilroy';
$where = 'here';
echo "$who was $where";
```

Kilroy was here

The other way is to surround the variable being interpolated with curly braces. Using this syntax ensures the correct variable is interpolated. The classic use of curly braces is to disambiguate the variable name from any surrounding text:

```
$n = 12;
```

echo "You are the {$n}th person";
You are the 12th person
Without the curly braces, PHP would try to print the value of the $nth variable.
Unlike in some shell environments, in PHP, strings are not repeatedly processed for interpolation. Instead, any interpolations in a double-quoted string are processed first and the result is used as the value of the string:

```
$bar = 'this is not printed';
$foo = '$bar'; // single quotes
print("$foo");
$bar
```

There are 4 ways to specify a string literal in PHP.

1. single quoted
2. double quoted
3. heredoc syntax
4. newdoc syntax (since PHP 5.3)

1.Single Quoted

We can create a string in PHP by enclosing the text in a single-quote. It is the easiest way to specify string in PHP.
For specifying a literal single quote, escape it with a backslash (\) and to specify a literal backslash (\) use double backslash (\\). All the other instances with backslash such as \r or \n, will be output same as they specified instead of having any special meaning.
Following some examples are given to understand the single quoted PHP String in a better way:
Example 1

```
1. <?php
2.     $str='Hello text within single quote';
3.     echo $str;
4. ?>
```

Output:

```
Hello text within single quote
```

We can store multiple line text, special characters, and escape sequences in a single-quoted PHP string.
Example 2

```
1. <?php
2. $str1='Hello text multiple line text within single quoted string';
3. $str2='Using double "quote" directly inside single quoted string';
4. $str3='Using escape sequences \n in single quoted string';
5. echo "$str1 <br/> $str2 <br/> $str3";
6. ?>
```

Output:

```
Hello text multiple line text within single quoted string
```

```
Using double "quote" directly inside single quoted string
Using escape sequences \n in single quoted string
```

Example 3

```
1. <?php
2. $num1=10;
3. $str1='trying variable $num1';
4. $str2='trying backslash n and backslash t inside single quoted string \n \t';
5. $str3='Using single quote \'my quote\' and \\backslash';
6. echo "$str1 <br/> $str2 <br/> $str3";
7. ?>
```

Output:

```
trying variable $num1
trying backslash n and backslash t inside single quoted string \n \t
Using single quote 'my quote' and \backslash
```

Note: In single quoted PHP strings, most escape sequences and variables will not be interpreted. But, we can use single quote through \' and backslash through \\ inside single quoted PHP strings.

2.Double Quoted

In PHP, we can specify string through enclosing text within double quote also. But escape sequences and variables will be interpreted using double quote PHP strings.

Example 1

```
1. <?php
2. $str="Hello text within double quote";
3. echo $str;
4. ?>
```

Output:

```
Hello text within double quote
```

Now, you **can't use double quote directly** inside double quoted string.

Example 2

```
1. <?php
2. $str1="Using double "quote" directly inside double quoted string";
3. echo $str1;
4. ?>
```

Output:

```
Parse error: syntax error, unexpected 'quote' (T_STRING) in C:\wamp\www\string1.php on line 2
```

We **can store multiple line text, special characters and escape sequences** in a double quoted PHP string.

Example 3

```
1. <?php
```

```
2. $str1="Hello text  multiple line  text within double quoted string";
3. $str2="Using double \"quote\" with backslash inside double quoted string";
4. $str3="Using escape sequences \n in double quoted string";
5. echo "$str1 <br/> $str2 <br/> $str3";
6. ?>
```

Output:

```
Hello text multiple line text within double quoted string
Using double "quote" with backslash inside double quoted string
Using escape sequences in double quoted string
```

In double quoted strings, **variable will be interpreted**.

Example 4

```
1. <?php
2. $num1=10;
3. echo "Number is: $num1";
4. ?>
```

Output:

```
Number is: 10
```

3.Heredoc

Heredoc syntax (<<<) is the third way to delimit strings. In Heredoc syntax, an identifier is provided after this heredoc <<< operator, and immediately a new line is started to write any text. To close the quotation, the string follows itself and then again that same identifier is provided. That closing identifier must begin from the new line without any whitespace or tab.

Naming Rules

The identifier should follow the naming rule that it must contain only alphanumeric characters and underscores, and must start with an underscore or a non-digit character.

For Example

Valid Example

```
1. <?php
2.     $str = <<<Demo
3. It is a valid example
4. Demo;    //Valid code as whitespace or tab is not valid before closing identifier
5. echo $str;
6. ?>
```

Output:

```
It is a valid example
```

Invalid Example

We cannot use any whitespace or tab before and after the identifier and semicolon, which means identifier must not be indented. The identifier must begin from the new line.

```
1. <?php
```

```
2.     $str = <<<Demo
3. It is Invalid example
4.         Demo;    //Invalid code as whitespace or tab is not valid before closing identifier
5. echo $str;
6. ?>
```

This code will generate an error.

Output:

Parse error: **syntax error, unexpected end of file in** C:\xampp\htdocs\xampp\PMA\heredoc.php on **line 7**

Heredoc is similar to the double-quoted string, without the double quote, means that quote in a heredoc are not required. It can also print the variable's value.

Example

```
1. <?php
2.     $city = 'Delhi';
3.     $str = <<<DEMO
4. Hello! My name is Misthi, and I live in $city.
5. DEMO;
6.     echo $str;
7.  ?>
```

Output:

Hello! My name is Misthi, and I live in Delhi.

Example

We can add multiple lines of text here between heredoc syntax.

```
1. <?php
2.     $str = <<<DEMO
3. It is the example
4. of multiple
5. lines of text.
6. DEMO;
7.     echo $str;
8.
9. echo '</br>';
10.
11. echo <<<DEMO    // Here we are not storing string content in variable str.
12. It is the example
13. of multiple
14. lines of text.
```

```
DEMO;
?>
```

Output:

```
It is the example of multiple lines of text.
It is the example of multiple lines of text.
```

Below are the example with class and their variable

Example

```
<?php
class heredocExample{
    var $demo;
    var $example;
    function __construct()
    {
        $this->demo = 'DEMO';
        $this->example = array('Example1', 'Example2', 'Example3');
    }
}
$heredocExample = new heredocExample();
$name =  'Gunjan';

echo <<<ECO
My name is "$name". I am printing some $heredocExample->demo example.
Now, I am printing {$heredocExample->example[1]}.
It will print a capital 'A': \x41
ECO;
?>
```

Output:

```
My name is "Gunjan". I am printing some DEMO example.
Now, I am printing Example2.
It will print a capital 'A': A
```

4.Newdoc

Newdoc is similar to the heredoc, but in newdoc parsing is not done. It is also identified with three less than symbols <<< followed by an identifier. But here identifier is enclosed in single-quote, **e.g. <<<'EXP'**. Newdoc follows the same rule as heredocs.

The difference between newdoc and heredoc is that - Newdoc is a **single-quoted string** whereas heredoc is a **double-quoted string**.

Note: Newdoc works as single quotes.

Example-1:

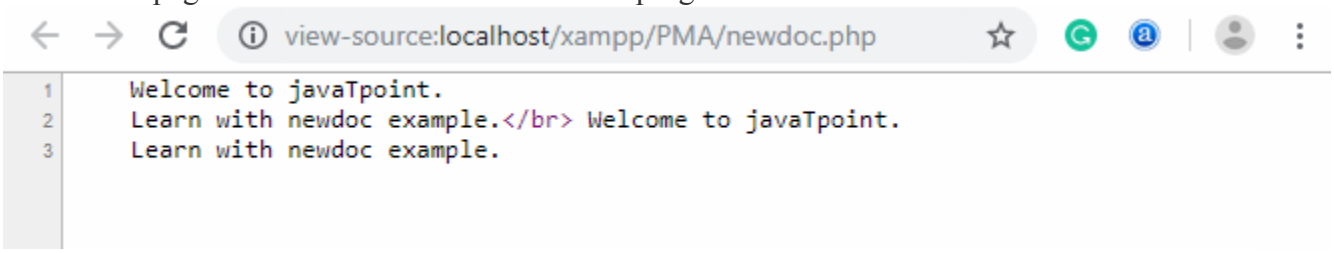
```
<?php
    $str = <<<'DEMO'
    Welcome to javaTpoint.
            Learn with newdoc example.
DEMO;
echo $str;
echo '</br>';

echo <<< 'Demo'    // Here we are not storing string content in variable str.
    Welcome to javaTpoint.
            Learn with newdoc example.
Demo;
?>
```

Output:

```
Welcome to javaTpoint. Learn with newdoc example.
Welcome to javaTpoint. Learn with newdoc example.
```

Go to view page source and see the source of the program.



Example

The below example shows that newdoc does not print the variable's value.

```
<?php
class heredocExample{
        var $demo;
        var $example;
        function __construct()
        {
                $this->demo = 'DEMO';
                $this->example = array('Example1', 'Example2', 'Example3');
        }
    }
    $heredocExample = new heredocExample();
```

```
12.     $name =  'Gunjan';
13.
14.     echo <<<ECO
15.     My name is "$name". I am printing some $heredocExample->demo example.
16.     Now, I am printing {$heredocExample->example[1]}.
17.     It will print a capital 'A': \x41
18. ECO;
19.  ?>
```

Output:

The output of the above program will be like:

```
My name is "$name". I am printing some $heredocExample->demo example.
Now, I am printing {$heredocExample->example[1]}.
It will print a capital 'A': \x41
```

Note: newdoc supported by PHP 5.3.0+ versions.

Invalid Example

We cannot use any whitespace or tab before and after the identifier and semicolon, means identifier must not be indented. The identifier must begin from the new line. It is also invalid in newdoc same as heredoc.

```
1.  <?php
2.      $str = <<<'Demo'
3.  It is Invalid example
4.  Demo;  //Invalid code as whitespace or tab is not valid before closing identifier
5.  echo $str;
6.  ?>
```

This code will generate an error.

Output:

```
Parse error: syntax error, unexpected end of file in C:\xampp\htdocs\xampp\PMA\newdoc.php
o
```

Printing Strings

There are four ways to send output to the browser. The echo construct lets you print many values at once, while print() prints only one value. The printf() function builds a formatted string by inserting values into a template. The print_r() function is useful for debugging—it prints the contents of arrays, objects, and other things, in a more-or-less human-readable form.

1.echo

To put a string into the HTML of a PHP-generated page, use echo. While it looks—and for the most part behaves—like a function, echo is a language construct. This means that you can omit the parentheses, so the following are equivalent:

```
echo "Printy";
echo("Printy"); // also valid
```

You can specify multiple items to print by separating them with commas:

```
echo "First", "second", "third";
Firstsecondthird
```

It is a parse error to use parentheses when trying to echo multiple values:

```
// this is a parse error
echo("Hello", "world");
```

Because echo is not a true function, you can't use it as part of a larger expression:

```
// parse error
if (echo("test")) {
  echo("It worked!");
}
```

Such errors are easily remedied, by using the print() or printf() functions.

2.print()

The print() construct sends one value (its argument) to the browser:

```
if (print("test\n")) {
  print("It worked!");
}
test
It worked!
```

3.printf()

The printf() function outputs a string built by substituting values into a template (the *format string*). It is derived from the function of the same name in the standard C library. The first

argument to printf() is the format string. The remaining arguments are the values to be substituted. A % character in the format string indicates a substitution.

Format modifiers

Specifier	Meaning
%	Displays the % character.
b	The argument is an integer and is displayed as a binary number.
c	The argument is an integer and is displayed as the character with that value.
d	The argument is an integer and is displayed as a decimal number.
e	The argument is a double and is displayed in scientific notation.
E	The argument is a double and is displayed in scientific notation using uppercase letters.
f	The argument is a floating-point number and is displayed as such in the current locale's format.
F	The argument is a floating-point number and is displayed as such.
g	The argument is a double and is displayed either in scientific notation (as with the %e type specifier) or as a floating-point number (as with the %f type specifier), whichever is shorter.
G	The argument is a double and is displayed either in scientific notation (as with the %E type specifier) or as a floating-point number (as with the %f type specifier), whichever is shorter.
o	The argument is an integer and is displayed as an octal (base-8) number.
s	The argument is a string and is displayed as such.
u	The argument is an unsigned integer and is displayed as a decimal number.
x	The argument is an integer and is displayed as a hexadecimal (base-16) number; lowercase letters are used.
X	The argument is an integer and is displayed as a hexadecimal (base-16) number; uppercase letters are used.

Each substitution marker in the template consists of a percent sign (%), possibly followed by modifiers from the following list, and ends with a type specifier. (Use %% to get a single percent character in the output.) The modifiers must appear in the order in which they are listed here:

- A padding specifier denoting the character to use to pad the results to the appropriate string size. Specify 0, a space, or any character prefixed with a single quote. Padding with spaces is the default.
- A sign. This has a different effect on strings than on numbers. For strings, a minus (-) here forces the string to be left-justified (the default is to right-justify). For numbers, a plus (+) here forces positive numbers to be printed with a leading plus sign (e.g., 35 will be printed as +35).
- The minimum number of characters that this element should contain. If the result would be less than this number of characters, the sign and padding specifier govern how to pad to this length.
- For floating-point numbers, a precision specifier consisting of a period and a number; this dictates how many decimal digits will be displayed. For types other than double, this specifier is ignored.

Type specifiers

The type specifier tells printf() what type of data is being substituted. This determines the interpretation of the previously listed modifiers. There are eight types, as listed in Table 4-2.

printf() type specifiers

The printf() function looks outrageously complex to people who aren't C programmers. Once you get used to it, though, you'll find it a powerful formatting tool. Here are some examples:

- A floating-point number to two decimal places:

- printf('%.2f', 27.452);

27.45

- Decimal and hexadecimal output:

- printf('The hex value of %d is %x', 214, 214);

The hex value of 214 is d6

- Padding an integer to three decimal places:

- printf('Bond. James Bond. %03d.', 7);

Bond. James Bond. 007.

- Formatting a date:

- printf('%02d/%02d/%04d', $month, $day, $year);
  02/15/2005

- A percentage:

- printf('%.2f%% Complete', 2.1);
  2.10% Complete

- Padding a floating-point number:

- printf('You\'ve spent $%5.2f so far', 4.1);
  You've spent $ 4.10 so far

The sprintf() function takes the same arguments as printf() but returns the built-up string instead of printing it. This lets you save the string in a variable for later use:

```
$date = sprintf("%02d/%02d/%04d", $month, $day, $year);
// now we can interpolate $date wherever we need a date
```

print_r() and var_dump()

The print_r() construct intelligently displays what is passed to it, rather than casting everything to a string, as echo and print() do. Strings and numbers are simply printed. Arrays appear as parenthesized lists of keys and values, prefaced by Array:

```
$a = array('name' => 'Fred', 'age' => 35, 'wife' => 'Wilma');
print_r($a);
Array
(
  [name] => Fred
  [age] => 35
  [wife] => Wilma)
```

Using print_r() on an array moves the internal iterator to the position of the last element in the array. See Chapter 5 for more on iterators and arrays.

When you print_r() an object, you see the word Object, followed by the initialized properties of the object displayed as an array:

```
class P {
  var $name = 'nat';
  // ...
}

$p = new P;
print_r($p);
Object
(
  [name] => nat)
```

Boolean values and NULL are not meaningfully displayed by print_r():

```
print_r(true); // prints "1";
1
print_r(false); // prints "";

print_r(null); // prints "";
```

For this reason, var_dump() is preferred over print_r() for debugging. The var_dump() function displays any PHP value in a human-readable format:

```
var_dump(true);
var_dump(false);
var_dump(null);
var_dump(array('name' => "Fred", 'age' => 35));
class P {
  var $name = 'Nat';
  // ...
}
$p = new P;
```

```
var_dump($p);
bool(true)
bool(false)
bool(null)
array(2) {
  ["name"]=>
  string(4) "Fred"
  ["age"]=>
  int(35)
}
object(p)(1) {
  ["name"]=>
  string(3) "Nat"
}
```

Beware of using print_r() or var_dump() on a recursive structure such as $GLOBALS (which has an entry for GLOBALS that points back to itself). The print_r() function loops infinitely, while var_dump() cuts off after visiting the same element three times.

Accessing Individual Characters

The strlen() function returns the number of characters in a string:

```
$string = 'Hello, world';

$length = strlen($string); // $length is 12
```

You can use the string offset syntax on a string to address individual characters:

```
$string = 'Hello';

for ($i=0; $i < strlen($string); $i++) {

  printf("The %dth character is %s\n", $i, $string{$i});

}
```

```
The 0th character is H

The 1th character is e
```

The 2th character is l

The 3th character is l

The 4th character is o

Cleaning Strings

Often, the strings we get from files or users need to be cleaned up before we can use them. Two common problems with raw data are the presence of extraneous whitespace and incorrect capitalization (uppercase versus lowercase).

Removing Whitespace

You can remove leading or trailing whitespace with the trim(), ltrim(), and rtrim() functions:

$trimmed = trim(*string* [, *charlist*]);

$trimmed = ltrim(*string* [, *charlist*]);

$trimmed = rtrim(*string* [, *charlist*]);

trim() returns a copy of *string* with whitespace removed from the beginning and the end. ltrim() (the *l* is for *left*) does the same, but removes whitespace only from the start of the string. rtrim() (the *r* is for *right*) removes whitespace only from the end of the string. The optional *charlist* argument is a string that specifies all the characters to strip. The default characters to strip are given in Table 4-3.

Table 4-3. Default characters removed by trim(), ltrim(), and rtrim()

Character	ASCII value	Meaning
" "	0x20	Space
"\t"	0x09	Tab
"\n"	0x0A	Newline (line feed)
"\r"	0x0D	Carriage return
"\0"	0x00	NUL-byte
"\x0B"	0x0B	Vertical tab

For example:

```
$title = "   Programming PHP  \n";

$str1 = ltrim($title);   // $str1 is "Programming PHP  \n"
```

```
$str2 = rtrim($title);   // $str2 is "   Programming PHP"

$str3 = trim($title);    // $str3 is "Programming PHP"
```

Given a line of tab-separated data, use the *charlist* argument to remove leading or trailing whitespace without deleting the tabs:

```
$record = "  Fred\tFlintstone\t35\tWilma\t   \n";

$record = trim($record, " \r\n\0\x0B ...
```

Encoding and Escaping

Because PHP programs often interact with HTML pages, web addresses (URLs), and databases, there are functions to help you work with those types of data. HTML, web page addresses, and database commands are all strings, but they each require different characters to be escaped in different ways. For instance, a space in a web address must be written as `%20`, while a literal less-than sign (`<`) in an HTML document must be written as `<`. PHP has a number of built-in functions to convert to and from these encodings.

HTML

Special characters in HTML are represented by *entities* such as `&` and `<`. There are two PHP functions that turn special characters in a string into their entities: one for removing HTML tags, and one for extracting only `meta` tags.

PHP Encoding for HTML

Sanitizing output for HTML

HTML can contain almost any character. There are a few characters which have special meanings in HTML and should be used with caution.

Reserved HTML characters

```
< > & "
```

These characters do not have special meanings in all parts of HTML. For example, it is not a problem to use " inside a paragraph of text, but it is a problem to use it inside an HTML attribute name.

```
<?php
  $title = 'Movie: "Star Wars"';
?>
```

```
<p title="<?php echo $title ?>">I love the movie "Star Wars".</p>
```

In the example above, the " is a problem for the title attribute but not for the content of the paragraph. Here is what the resulting HTML would look like. HTML would read the title value as "Movie: ".

```
<p title="Movie: "Star Wars"">I love the movie "Star Wars".</p>
```

The < and > are the most problematic characters because they indicate the start and end of tags. Using them accidentally could open a new tag or close an existing tag when it is not intended and break the entire page structure.
Allowing these characters in dynamic data also opens up the possibility that additional tags—including form elements and JavaScript—could be inserted in the HTML of the page. This is a common security vulnerability that hackers like to exploit.

HTML encoding

All dynamic values should be encoded (i.e. "transformed") before being used anywhere in HTML. This will ensure that the content does not interfere with the structure of the HTML. Web developers output a lot of dynamic data to HTML, so HTML encoding happens routinely.

This is a major concern for security because embedded-JavaScript needs HTML tags to function. It will be a primary defense against Cross-Site Scripting attacks.

There are different types of encoding depending on the context. Encoding for HTML means converting reserved characters into HTML character entities.

HTML character entities are written as &code;, where "code" is an abbreviation or a number to represent each character. There are thousands of HTML character entities, but for encoding special characters, there are only four that matter.

Char	**Entity**
<	<
>	>
&	&

=	"

PHP encoding functions

PHP has two built-in functions which can help with HTML encoding. The first encodes only the four reserved characters. The second encodes as much as it can.

htmlspecialchars()

- Encode reserved characters as HTML entities
- Ignores single quotes by default, but configurable
- Use for *all* output inside HTML

htmlentities()

- Encode all possible characters as HTML entities
- Use for safe and pretty output in HTML

Example:

```
<?php $string = 'We have to watch out for < and & as well as " and >'; ?>
<p><?php echo htmlspecialchars($string); ?></p>
```

Would output:

```
<p>We have to watch out for &amp;lt; and &amp;amp; as well as &amp;quot; and
&amp;gt;</p>
```

Example:

```
<?php $symbols = "™ ® © • £ ¢ ¥"; ?>
<p><?php echo 'Symbols: ' . htmlentities($symbols); ?></p>
```

Would output:

```
<p>Symbols: &amp;trade; &amp;reg; &amp;copy; &amp;bull; &amp;pound;
&amp;cent; &amp;yen;</p>
```

There are PHP functions which can decode these encoded strings (*htmlspecialchars_decode()*, *html_entity_decode()*) but they are almost never needed because the browser does the decoding that matters when it processes the HTML page.

Pro Tip

Because encoding for HTML is done frequently and because the function name is very long, most PHP developers define a custom function as a short cut.

```
<?php
  function h($string="") {
    return htmlspecialchars($string);
  }
?>

<?php echo h("This is safe for < and >."); ?>
```

Much easier!

Encoding for URLs inside HTML

When outputing a dynamic link to an HTML page, it should be encoded for the URL and also encoded for HTML. Because **all output** should be encoded for HTML.

Example:

```
<?php
  $course = 'web security';
  $query = 'URL encode & decode';
  $label = 'Link label with < and >';

  $url = rawurlencode('/courses/' . $course . '/content');
  $url .= '?search=' . urlencode($query);
?>

<a href="<?php echo htmlspecialchars($url); ?>">
  <?php echo htmlspecialchars($label); ?>
</a>
```

Other HTMl sanitizing functions

strip_tags()

PHP's strip_tags() function will remove all HTML and PHP tags from a string. It is an exception to the "don't remove content" rule because it is well-designed to remove *all* tags.

```
<?php
 $string = '<p>Text</p><!-- Comment --><a href="link.php">Link</a>';
 echo strip_tags($string);
 // TextLink
?>
```

It is possible to whitelist tags which should still be allowed but, as the PHP manual notes, this opens up the possibility for abuse of the tag attributes such as style and onmouseover.

When removing all tags, *strip_tags()* is as secure as *htmlspecialchars()*. When tags are allowed *strip_tags()* is much less secure than *htmlspecialchars()* and should be used with caution.

How do I compare strings in PHP?

The easiest way to compare PHP strings is to use the logical operators ('==') and ('!='). If you want to know which string is lexicographically greater or lesser, you can use the operators ('<'), ('>'), ('<='), ('>='). You can also compare strings using the strcmp() and strcasecmp() functions. The strcmp() function is a case-sensitive safe binary string comparison function that returns 0 if the strings match. The strcasecmp() function is similar but performs a case-insensitive comparison. In this PHP String Compare example, we compare strings using the ("==") operator. Click Execute to run the PHP Compare Strings Example online and see the result.

How do I compare strings in PHP?Execute

```
<?php
$string1 = 'first string';
$string2 = 'second string';

if ($string1 == $string2) {
        echo 'Strings are equal';
}
else {
        echo 'Strings are not equal';
}
```

```
?>
```

Updated: Jan 30, 2022 Viewed: 1774 times
Author: ReqBin

Comparing PHP strings using the logical operators

You can use the logical operators ("==") and ("!=") to compare two strings. You can use the ("==") operator to check if strings are equal and the ("!=") operator to check if strings are not equal.

Comparing two strings using "==" operator

```
<?php
$string1 = 'first string';
$string2 = 'second string';

if ($string1 == $string2) {
   echo 'Strings are equal';
}
else {
   echo 'Strings are not equal';
}
?>

#output: Strings are not equal
```

Comparing PHP strings using less than or greater than logical operators

You can compare ("<") or (">") comparisons between strings in PHP, as well as between numbers. If two strings start the same and then differ, then the string with the first character that differs will be smaller. This order is called lexicographic (the same is used in dictionaries).

PHP string comparison with '<' or '>' operators

```
<?php
$string1 = 'abc';
$string2 = 'abcd';

if ($string1 > $string2) {
   echo 'true';
}
else {
   echo 'false';
}
?>

#output: false
```

How то check if two strings are the same in PHP?

To compare two strings in PHP, you can use the strcmp() function. This function compares two strings character-by-character (more precisely, byte-by-byte) and returns:

0: - if the strings completely match;
-1: - if string str1 is lexicographically less than str2;
1: - if, on the contrary, str1 is greater than str2.

Since the comparison is byte-by-byte, the case of characters affects the results of comparisons.

PHP strcmp() Syntax

```
strcmp(str1, str2)
```

Where:

- str1 and str2 (Required): specifies the first and second string to compare

PHP strcmp() Example

```
<?php
echo strcmp('I like PHP', 'I like PHP');
?>

#output: 0
```

How to compare strings case insensitive in PHP?

To compare strings case-insensitively in PHP, you can use the strcasecmp() function. The function differs from strcmp() in that it does not compare the whole world but the first length bytes. If length is less than the length of the smallest of the strings, then the strings are compared as a whole. The strcasecmp() function compares two strings character-by-character (more precisely, byte-by-byte) and returns:

0: - if the strings match exactly;
-1: - if string str1 is lexicographically less than str2;
1: - if, on the contrary, str1 is "greater than" str2.

Since the comparison is byte-by-byte, the case of characters affects the results of comparisons.

PHP strcasecmp() Syntax

```
strncmp(str1, str2)
```

Where:

- str1 and str2 (Required): specifies the first and second string to compare

PHP strcasecmp() Example

```
<?php
echo strcasecmp('Pineapple', 'pineapple');
?>

#output: 0
```

A variation on strcmp() is strcasecmp(), which converts strings to lowercase before comparing them. Its arguments and return values are the same as those for strcmp():
$n = strcasecmp("Fred", "frED"); // $n is 0
Another variation on string comparison is to compare only the first few characters of the string. The strncmp() and strncasecmp() functions take an additional argument, the initial number of characters to use for the comparisons:
$relationship = strncmp(string_1, string_2, len);
$relationship = strncasecmp(string_1, string_2, len);

The escape-sequence replacements are:

- \n is replaced by the newline character
- \r is replaced by the carriage-return character
- \t is replaced by the tab character
- \$ is replaced by the dollar sign itself ($)
- \" is replaced by a single double-quote (")
- \\ is replaced by a single backslash (\)

Here's an example to clarify the differences between single and double quoted strings:

Example

```
<?php
$my_str = 'World';
echo "Hello, $my_str!<br>";      // Displays: Hello World!
echo 'Hello, $my_str!<br>';      // Displays: Hello, $my_str!

echo '<pre>Hello\tWorld!</pre>'; // Displays: Hello\tWorld!
echo "<pre>Hello\tWorld!</pre>"; // Displays: Hello    World!
echo 'I\'ll be back';            // Displays: I'll be back
?>
```

Manipulating PHP Strings

PHP provides many built-in functions for manipulating strings like calculating the length of a string, find substrings or characters, replacing part of a string with different

characters, take a string apart, and many others. Here are the examples of some of these functions.

Calculating the Length of a String
The strlen() function is used to calculate the number of characters inside a string. It also includes the blank spaces inside the string.

Example

```
<?php
$my_str = 'Welcome to Tutorial Republic';

// Outputs: 28
echo strlen($my_str);
?>
```

Counting Number of Words in a String
The str_word_count() function counts the number of words in a string.

Example

```
<?php
$my_str = 'The quick brown fox jumps over the lazy dog.';

// Outputs: 9
echo str_word_count($my_str);
?>
```

Replacing Text within Strings
The str_replace() replaces all occurrences of the search text within the target string.

Example

```
<?php
$my_str = 'If the facts do not fit the theory, change the facts.';

// Display replaced string
echo str_replace("facts", "truth", $my_str);
```

```
?>
```

The output of the above code will be:
If the truth do not fit the theory, change the truth.
You can optionally pass the fourth argument to the str_replace() function to know how many times the string replacements was performed, like this.

Example

```
<?php
$my_str = 'If the facts do not fit the theory, change the facts.';

// Perform string replacement
str_replace("facts", "truth", $my_str, $count);

// Display number of replacements performed
echo "The text was replaced $count times.";
?>
```

The output of the above code will be:
The text was replaced 2 times.

Reversing a String
The strrev() function reverses a string.

Example

```
<?php
$my_str = 'You can do anything, but not everything.';

// Display reversed string
echo strrev($my_str);
?>
```

The output of the above code will be:
.gnihtyreve ton tub ,gnihtyna od nac uoY

Searching Strings in PHP

When writing PHP scripts, you often need to search a string for a particular chunk of text. For example, you might be writing a search engine to search through pages of content, or you might want to know if a URL or email address contains a certain domain name.
PHP gives you many useful functions for searching strings. In this article you look at:

- strstr() for finding out whether some text is in a string

- strpos() and strrpos() for finding the position of some text in a string
- substr_count() for finding out how many times some text appears in a string

Simple text searching with strstr()

PHP's strstr() function simply takes a string to search, and a chunk of text to search for. If the text was found, it returns the portion of the string from the first character of the match up to the end of the string:

```
$myString = 'Hello, there!';
echo strstr( $myString, 'llo' );  // Displays "llo, there!"
```

If the text wasn't found then strstr() returns false. You can use this fact to determine if the text chunk was in the string or not:

```
$myString = 'Hello, there!';
if ( strstr( $myString, 'Goodbye' ) ) {
  echo "Text found";
} else {
  echo "Text not found";
}
```

The above code displays:

```

Text not found
```

strstr() is *case sensitive* — for example, "hello" won't match "Hello". If you don't care about matching case, use the case-insensitive version, stristr(), instead.

strrpos()

strpos() takes the same 2 arguments as strstr(). However, rather than returning a portion of the string, it returns the index of the first character of the matched text in the string:

```
$myString = 'Hello, there!';
echo strpos( $myString, 'llo' );  // Displays "2"
```

In the above code, strpos() finds the text 'llo' in the target string starting at the 3rd character, so it returns 2. (Remember that character index positions in strings start from 0, not 1.)

Be careful when using strpos() to check for the existence of a match. The following code incorrectly displays "Not found", because strpos() returns 0, which is equivalent to false in PHP:

```
$myString = 'Hello, there!';
if ( strpos( $myString, 'Hello' ) ) {
  echo "Found";
} else {
  echo "Not found";
}
```

To fix this, make sure you test explicitly for false by using the === or !== operator. The following code correctly displays "Found":

```
$myString = 'Hello, there!';
if ( strpos( $myString, 'Hello' ) !== false ) {
  echo "Found";
} else {
  echo "Not found";
}
```

You can pass a third argument to strpos() to specify the index position from which to begin the search:

```
$myString = 'Hello, there!';
echo strpos( $myString, 'e' ) . '<br />';     // Displays "1"
echo strpos( $myString, 'e', 2 ) . '<br />';  // Displays "9"
```

The strrpos() function is very similar to strpos(). The only difference is that it finds the *last* match in the string instead of the first:

```
$myString = 'Hello, there!';
echo strpos( $myString, 'e' ) . "<br />";   // Displays "1"
echo strrpos( $myString, 'e' ) . "<br />";  // Displays "11"
```

As with strstr(), the strpos() and strrpos() functions are case sensitive. Their case-insensitive equivalents are stripos() and strripos() respectively.

Counting matches with substr_count()

You can use PHP's substr_count() function to find the number of times a chunk of text appears in the target string:

```
$myString = 'Short stories';
echo substr_count( $myString, 'or' );  // Displays "2"
```

As with strpos() and strrpos(), you can pass an optional third argument: the index position to begin the search. For example:

```
$myString = 'Short stories';
echo substr_count( $myString, 'or', 6 );  // Displays "1"
```

You can also pass an optional fourth argument: the number of characters after the offset position in which to search for the text. For example:

```
echo substr_count( $myString, 'or', 0, 10 ) . '<br />';  // Displays "2"
echo substr_count( $myString, 'or', 0, 5 ) . '<br />';   // Displays "1"
```

Searches using masks

If you thought strrchr() was esoteric, you haven't seen anything yet. The strspn() and strcspn() functions tell you how many characters at the beginning of a string are composed of certain characters:

$length = strspn(string, charset);

For example, this function tests whether a string holds an octal number:

```
function isOctal($str)
{
return strspn($str, '01234567') == strlen($str);
}
```

The c in strcspn() stands for complement—it tells you how much of the start of the string is not composed of the characters in the character set. Use it when the number of interesting characters is greater than the number of uninteresting characters. For example, this function tests whether a string has any NUL-bytes, tabs, or carriage returns:

```
function hasBadChars($str)
{
return strcspn($str, "\n\t\0") != strlen($str);
}
```

Decomposing URLs

The parse_url() function returns an array of components of a URL:

```
$array = parse_url(url);
```

For example:

```
$bits = parse_url("http://me:secret@example.com/cgi-bin/board?user=fred");
print_r($bits);
Array
(
[scheme] => http
[host] => example.com
[user] => me
[pass] => secret
[path] => /cgi-bin/board
[query] => user=fred)
```

The possible keys of the hash are scheme, host, port, user, pass, path, query, and fragment.

Regular expressions

A regular expression is a sequence of characters that forms a search pattern. When you search for data in a text, you can use this search pattern to describe what you are searching for.

A regular expression can be a single character, or a more complicated pattern.

Regular expressions can be used to perform all types of text search and text replace operations.

Syntax

In PHP, regular expressions are strings composed of delimiters, a pattern and optional modifiers.

$exp = "/w3schools/i";

In the example above, / is the **delimiter**, *w3schools* is the **pattern** that is being searched for, and i is a **modifier** that makes the search case-insensitive.

The delimiter can be any character that is not a letter, number, backslash or space. The most common delimiter is the forward slash (/), but when your pattern contains forward slashes it is convenient to choose other delimiters such as # or ~.

Regular Expression Functions

PHP provides a variety of functions that allow you to use regular expressions.
The preg_match(), preg_match_all() and preg_replace() functions are some of the most commonly used ones:

Function	**Description**
preg_match()	Returns 1 if the pattern was found in the string and 0 if not
preg_match_all()	Returns the number of times the pattern was found in the string, which may also be 0
preg_replace()	Returns a new string where matched patterns have been replaced with another string

Using preg_match()

The preg_match() function will tell you whether a string contains matches of a pattern.

Example

Use a regular expression to do a case-insensitive search for "w3schools" in a string:

```
<?php
$str = "Visit W3Schools";
$pattern = "/w3schools/i";
echo preg_match($pattern, $str); // Outputs 1
?>
```

Using preg_match_all()

The preg_match_all() function will tell you how many matches were found for a pattern in a string.

Example

Use a regular expression to do a case-insensitive count of the number of occurrences of "ain" in a string:

```
<?php
$str = "The rain in SPAIN falls mainly on the plains.";
$pattern = "/ain/i";
echo preg_match_all($pattern, $str); // Outputs 4
?>
```

Using preg_replace()

The preg_replace() function will replace all of the matches of the pattern in a string with another string.

Example

Use a case-insensitive regular expression to replace Microsoft with W3Schools in a string:

```php
<?php
$str = "Visit Microsoft!";
$pattern = "/microsoft/i";
echo preg_replace($pattern, "W3Schools", $str); // Outputs "Visit W3Schools!"
?>
```

Regular Expression Modifiers

Modifiers can change how a search is performed.

Modifier	Description
i	Performs a case-insensitive search
m	Performs a multiline search (patterns that search for the beginning or end of a string will match the beginning or end of each line)
u	Enables correct matching of UTF-8 encoded patterns

Regular Expression Patterns

Brackets are used to find a range of characters:

Expression	Description
[abc]	Find one character from the options between the brackets

[^abc]	Find any character NOT between the brackets
[0-9]	Find one character from the range 0 to 9

Metacharacters

Metacharacters are characters with a special meaning:

Metacharacter	**Description**
|	Find a match for any one of the patterns separated by | as in: cat|dog|fish
.	Find just one instance of any character
^	Finds a match as the beginning of a string as in: ^Hello
$	Finds a match at the end of the string as in: World$
\d	Find a digit
\s	Find a whitespace character
\b	Find a match at the beginning of a word like this: \bWORD, or at

	the end of a word like this: WORD\b
\uxxxx	Find the Unicode character specified by the hexadecimal number xxxx

Quantifiers

Quantifiers define quantities:

Quantifier	**Description**
n+	Matches any string that contains at least one *n*
n*	Matches any string that contains zero or more occurrences of *n*
n?	Matches any string that contains zero or one occurrences of *n*
n{x}	Matches any string that contains a sequence of *X* *n*'s
n{x,y}	Matches any string that contains a sequence of X to Y *n*'s
n{x,}	Matches any string that contains a sequence of at least X *n*'s

Note: If your expression needs to search for one of the special characters you can use a backslash (\ ) to escape them. For example, to search for one or more question marks you can use the following expression: $pattern = '/\?+/';

Grouping

You can use parentheses ( ) to apply quantifiers to entire patterns. They also can be used to select parts of the pattern to be used as a match.

Example

Use grouping to search for the word "banana" by looking for *ba* followed by two instances of *na*:

```
<?php
$str = "Apples and bananas.";
$pattern = "/ba(na){2}/i";
echo preg_match($pattern, $str); // Outputs 1
?>
```

PHP Arrays

An array stores multiple values in one single variable:

Example

```
<?php
$cars = array("Volvo", "BMW", "Toyota");
echo "I like " . $cars[0] . ", " . $cars[1] . " and " . $cars[2] . ".";
?>
```

- An array is a special variable, which can hold more than one value at a time.
- If you have a list of items (a list of car names, for example), storing the cars in single variables could look like this:

```
$cars1 = "Volvo";
$cars2 = "BMW";
$cars3 = "Toyota";
```

- However, what if you want to loop through the cars and find a specific one? And what if you had not 3 cars, but 300?
- The solution is to create an array!
- An array can hold many values under a single name, and you can access the values by referring to an index number.

Create an Array in PHP

In PHP, the array() function is used to create an array:

array();

In PHP, there are three types of arrays:

- **Indexed arrays** - Arrays with a numeric index
- **Associative arrays** - Arrays with named keys
- **Multidimensional arrays** - Arrays containing one or more arrays

Get The Length of an Array - The count() Function

The count() function is used to return the length (the number of elements) of an array:

Example

```
<?php
$cars = array("Volvo", "BMW", "Toyota");
echo count($cars);
?>
```

PHP Indexed Arrays

There are two ways to create indexed arrays:

The index can be assigned automatically (index always starts at 0), like this:

$cars = array("Volvo", "BMW", "Toyota");

or the index can be assigned manually:

```
$cars[0] = "Volvo";
$cars[1] = "BMW";
$cars[2] = "Toyota";
```

The following example creates an indexed array named $cars, assigns three elements to it, and then prints a text containing the array values:

Example

```
<?php
$cars = array("Volvo", "BMW", "Toyota");
```

```
echo "I like " . $cars[0] . ", " . $cars[1] . " and " . $cars[2] . ".";
?>
```

Loop Through an Indexed Array

To loop through and print all the values of an indexed array, you could use a for loop, like this:

Example

```
<?php
$cars = array("Volvo", "BMW", "Toyota");
$arrlength = count($cars);

for($x = 0; $x < $arrlength; $x++) {
  echo $cars[$x];
  echo "<br>";
}
?>
```

PHP Associative Arrays

Associative arrays are arrays that use named keys that you assign to them.

There are two ways to create an associative array:

```
$age = array("Peter"=>"35", "Ben"=>"37", "Joe"=>"43");
```

or:

```
$age['Peter'] = "35";
$age['Ben'] = "37";
$age['Joe'] = "43";
```

The named keys can then be used in a script:

Example

```
<?php
$age = array("Peter"=>"35", "Ben"=>"37", "Joe"=>"43");
echo "Peter is " . $age['Peter'] . " years old.";
```

```
?>
```

Loop Through an Associative Array

To loop through and print all the values of an associative array, you could use a foreach loop, like this:

Example

```
<?php
$age = array("Peter"=>"35", "Ben"=>"37", "Joe"=>"43");

foreach($age as $x => $x_value) {
  echo "Key=" . $x . ", Value=" . $x_value;
  echo "<br>";
}
?>
```

PHP - Multidimensional Arrays

A multidimensional array is an array containing one or more arrays.

PHP supports multidimensional arrays that are two, three, four, five, or more levels deep. However, arrays more than three levels deep are hard to manage for most people.

The dimension of an array indicates the number of indices you need to select an element.

- For a two-dimensional array you need two indices to select an element
- For a three-dimensional array you need three indices to select an element

PHP - Two-dimensional Arrays

A two-dimensional array is an array of arrays (a three-dimensional array is an array of arrays of arrays).

First, take a look at the following table:

Name	Stock	Sold
Volvo	22	18
BMW	15	13
Saab	5	2
Land Rover	17	15

We can store the data from the table above in a two-dimensional array, like this:

```
$cars = array (
  array("Volvo",22,18),
  array("BMW",15,13),
  array("Saab",5,2),
  array("Land Rover",17,15)
);
```

Now the two-dimensional $cars array contains four arrays, and it has two indices: row and column.

To get access to the elements of the $cars array we must point to the two indices (row and column):

Example

```
<?php
echo $cars[0][0].": In stock: ".$cars[0][1].", sold: ".$cars[0][2].".<br>";
echo $cars[1][0].": In stock: ".$cars[1][1].", sold: ".$cars[1][2].".<br>";
echo $cars[2][0].": In stock: ".$cars[2][1].", sold: ".$cars[2][2].".<br>";
echo $cars[3][0].": In stock: ".$cars[3][1].", sold: ".$cars[3][2].".<br>";
```

```
?>
```

We can also put a for loop inside another for loop to get the elements of the $cars array (we still have to point to the two indices):

Example

```
<?php
for ($row = 0; $row < 4; $row++) {
  echo "<p><b>Row number $row</b></p>";
  echo "<ul>";
  for ($col = 0; $col < 3; $col++) {
    echo "<li>".$cars[$row][$col]."</li>";
  }
  echo "</ul>";
}
?>
```

Extracting Multiple Values

PHP list() Function

Example

Assign variables as if they were an array:

```
<?php
$my_array = array("Dog","Cat","Horse");

list($a, $b, $c) = $my_array;
echo "I have several animals, a $a, a $b and a $c.";
?>
```

Definition and Usage

The list() function is used to assign values to a list of variables in one operation.

Note: Prior to PHP 7.1, this function only worked on numerical arrays.

Syntax

list(*var1, var2, ...*)

Parameter Values

Parameter	Description
var1	Required. The first variable to assign a value to
var2,...	Optional. More variables to assign values to

Technical Details

Return Value:	Returns the assigned array
PHP Version:	4+

More Examples

Example
Using the first and third variables:

```
<?php
$my_array = array("Dog","Cat","Horse");

list($a, , $c) = $my_array;
echo "Here I only use the $a and $c variables.";
?>
```

Slicing an Array

PHP array_slice() Function

Example
Start the slice from the third array element, and return the rest of the elements in the array:

```
<?php
$a=array("red","green","blue","yellow","brown");
print_r(array_slice($a,2));
?>
```

Definition and Usage

The array_slice() function returns selected parts of an array.
Note: If the array have string keys, the returned array will always preserve the keys (See example 4).

Syntax
array_slice(*array, start, length, preserve*)
Parameter Values

Parameter	Description
array	Required. Specifies an array
start	Required. Numeric value. Specifies where the function will start the

	slice. 0 = the first element. If this value is set to a negative number, the function will start slicing that far from the last element. -2 means start at the second last element of the array.
length	Optional. Numeric value. Specifies the length of the returned array. If this value is set to a negative number, the function will stop slicing that far from the last element. If this value is not set, the function will return all elements, starting from the position set by the start-parameter.
preserve	Optional. Specifies if the function should preserve or reset the keys. Possible values: • true - Preserve keys • false - Default. Reset keys

Technical Details

Return Value:	Returns selected parts of an array
PHP Version:	4+
PHP Changelog:	The *preserve* parameter was added in PHP 5.0.2

More Examples

Example 1

Start the slice from from the second array element, and return only two elements:

```
<?php
$a=array("red","green","blue","yellow","brown");
print_r(array_slice($a,1,2));
?>
```

Example 2

Using a negative start parameter:

```
<?php
$a=array("red","green","blue","yellow","brown");
print_r(array_slice($a,-2,1));
?>
```

Example 3

With the preserve parameter set to true:

```
<?php
$a=array("red","green","blue","yellow","brown");
print_r(array_slice($a,1,2,true));
```

```
?>
```

Example 4
With both string and integer keys:

```
<?php
$a=array("a"=>"red","b"=>"green","c"=>"blue","d"=>"yellow","e"=>"brown");
print_r(array_slice($a,1,2));

$a=array("0"=>"red","1"=>"green","2"=>"blue","3"=>"yellow","4"=>"brown");
print_r(array_slice($a,1,2));
?>
```

Splitting an Array into Chunks

PHP array_chunk() Function

Example
Split an array into chunks of two:

```
<?php
$cars=array("Volvo","BMW","Toyota","Honda","Mercedes","Opel");
print_r(array_chunk($cars,2));
?>
```

Definition and Usage
The array_chunk() function splits an array into chunks of new arrays.

Syntax
array_chunk(*array*, *size*, *preserve_key*)
Parameter Values

Parameter	Description
array	Required. Specifies the array to use
size	Required. An integer that specifies the size of each chunk
preserve_key	Optional. Possible values: • true - Preserves the keys • false - Default. Reindexes the chunk numerically

Technical Details

Return Value:	Returns a multidimensional indexed array, starting with zero, with each dimension containing *size* elements
PHP Version:	4.2+

More Examples

Example

Split an array into chunks of two and preserve the original keys:

```
<?php
$age=array("Peter"=>"35","Ben"=>"37","Joe"=>"43","Harry"=>"50");
print_r(array_chunk($age,2,true));
?>
```

Keys

PHP array_keys() Function

Example

Return an array containing the keys:

```
<?php
$a=array("Volvo"=>"XC90","BMW"=>"X5","Toyota"=>"Highlander");
print_r(array_keys($a));
?>
```

Definition and Usage

The array_keys() function returns an array containing the keys.

Syntax

array_keys(*array, value, strict*)

Parameter Values

Parameter	Description
array	Required. Specifies an array
value	Optional. You can specify a value, then only the keys with this value are returned
strict	Optional. Used with the value parameter. Possible values: • true - Returns the keys with the specified value, depending on type: the number 5 is not the same as the string "5". • false - Default value. Not depending on type, the number 5 is the same as the string "5".

Technical Details

Return Value:	Returns an array containing the keys
PHP Version:	4+

Changelog:	The *strict* parameter was added in PHP 5.0

More Examples

Example
Using the value parameter:

```
<?php
$a=array("Volvo"=>"XC90","BMW"=>"X5","Toyota"=>"Highlander");
print_r(array_keys($a,"Highlander"));
?>
```

Example
Using the strict parameter, false:

```
<?php
$a=array(10,20,30,"10");
print_r(array_keys($a,"10",false));
?>
```

Example
Using the strict parameter, true:

```
<?php
$a=array(10,20,30,"10");
print_r(array_keys($a,"10",true));
?>
```

Values

PHP array_values() Function

Example
Return all the values of an array (not the keys):

```
<?php
$a=array("Name"=>"Peter","Age"=>"41","Country"=>"USA");
print_r(array_values($a));
?>
```

Definition and Usage

The array_values() function returns an array containing all the values of an array.
Tip: The returned array will have numeric keys, starting at 0 and increase by 1.

Syntax
array_values(*array*)
Parameter Values

Parameter	Description
array	Required. Specifying an array

Technical Details

Return Value:	Returns an array containing all the values of an array
PHP Version:	4+

Checking Whether an Element Exists
PHP array_key_exists() Function

Example
Check if the key "Volvo" exists in an array:

```
<?php
$a=array("Volvo"=>"XC90","BMW"=>"X5");
if (array_key_exists("Volvo",$a))
  {
  echo "Key exists!";
  }
else
  {
  echo "Key does not exist!";
  }
?>
```

Definition and Usage
The array_key_exists() function checks an array for a specified key, and returns true if the key exists and false if the key does not exist.
Tip: Remember that if you skip the key when you specify an array, an integer key is generated, starting at 0 and increases by 1 for each value. (See example below)

Syntax
array_key_exists(*key, array*)
Parameter Values

Parameter	Description
key	Required. Specifies the key
array	Required. Specifies an array

Technical Details

Return Value:	Returns TRUE if the key exists and FALSE if the key does not exist

PHP Version: 4.0.7+

More Examples

Example

Check if the key "Toyota" exists in an array:

```
<?php
$a=array("Volvo"=>"XC90","BMW"=>"X5");
if (array_key_exists("Toyota",$a))
  {
  echo "Key exists!";
  }
else
  {
  echo "Key does not exist!";
  }
?>
```

Example

Check if the integer key "0" exists in an array:

```
<?php
$a=array("Volvo","BMW");
if (array_key_exists(0,$a))
  {
  echo "Key exists!";
  }
else
  {
  echo "Key does not exist!";
  }
?>
```

Removing and Inserting Elements in an Array

PHP array_splice() Function

Example

Remove elements from an array and replace it with new elements:

```
<?php
$a1=array("a"=>"red","b"=>"green","c"=>"blue","d"=>"yellow");
$a2=array("a"=>"purple","b"=>"orange");
array_splice($a1,0,2,$a2);
print_r($a1);
?>
```

Definition and Usage
The array_splice() function removes selected elements from an array and replaces it with new elements. The function also returns an array with the removed elements.
Tip: If the function does not remove any elements (length=0), the replaced array will be inserted from the position of the start parameter (See Example 2).
Note: The keys in the replaced array are not preserved.

Syntax
array_splice(*array, start, length, array*)
Parameter Values

Parameter	Description
array	Required. Specifies an array
start	Required. Numeric value. Specifies where the function will start removing elements. 0 = the first element. If this value is set to a negative number, the function will start that far from the last element. -2 means start at the second last element of the array.
length	Optional. Numeric value. Specifies how many elements will be removed, and also length of the returned array. If this value is set to a negative number, the function will stop that far from the last element. If this value is not set, the function will remove all elements, starting from the position set by the start-parameter.
array	Optional. Specifies an array with the elements that will be inserted to the original array. If it's only one element, it can be a string, and does not have to be an array.

Technical Details

Return Value:	Returns the array consisting of the extracted elements
PHP Version:	4+

More Examples
Example 1
The same example as the example on top of the page, but the output is the returned array:

```
<?php
$a1=array("a"=>"red","b"=>"green","c"=>"blue","d"=>"yellow");
```

```
$a2=array("a"=>"purple","b"=>"orange");
print_r(array_splice($a1,0,2,$a2));
?>
```

Example 2
With the length parameter set to 0:

```
<?php
$a1=array("0"=>"red","1"=>"green");
$a2=array("0"=>"purple","1"=>"orange");
array_splice($a1,1,0,$a2);
print_r($a1);
?>
```

Creating Variables from an Array

PHP extract() Function

Example
Assign the values "Cat", "Dog" and "Horse" to the variables $a, $b and $c:

```
<?php
$a = "Original";
$my_array = array("a" => "Cat","b" => "Dog", "c" => "Horse");
extract($my_array);
echo "\$a = $a; \$b = $b; \$c = $c";
?>
```

Definition and Usage
The extract() function imports variables into the local symbol table from an array.
This function uses array keys as variable names and values as variable values. For each element it will create a variable in the current symbol table.
This function returns the number of variables extracted on success.

Syntax
extract(*array, extract_rules, prefix*)
Parameter Values

Parameter	Description
array	Required. Specifies the array to use
extract_rules	Optional. The extract() function checks for invalid variable names and collisions with existing variable names. This parameter specifies how invalid and colliding names are treated. Possible values: • EXTR_OVERWRITE - Default. On collision, the existing variable is overwritten

	• EXTR_SKIP - On collision, the existing variable is not overwritten • EXTR_PREFIX_SAME - On collision, the variable name will be given a prefix • EXTR_PREFIX_ALL - All variable names will be given a prefix • EXTR_PREFIX_INVALID - Only invalid or numeric variable names will be given a prefix • EXTR_IF_EXISTS - Only overwrite existing variables in the current symbol table, otherwise do nothing • EXTR_PREFIX_IF_EXISTS - Only add prefix to variables if the same variable exists in the current symbol table • EXTR_REFS - Extracts variables as references. The imported variables are still referencing the values of the array parameter
prefix	Optional. If EXTR_PREFIX_SAME, EXTR_PREFIX_ALL, EXTR_PREFIX_INVALID or EXTR_PREFIX_IF_EXISTS are used in the extract_rules parameter, a specified prefix is required. This parameter specifies the prefix. The prefix is automatically separated from the array key by an underscore character.

Technical Details

Return Value:	Returns the number of variables extracted on success
PHP Version:	4+
PHP Changelog:	The *extract_rules* value EXTR_REFS was added in PHP 4.3. The *extract_rules* values EXTR_IF_EXISTS and EXTR_PREFIX_IF_EXISTS were added in PHP 4.2. As of PHP 4.0.5, this function now returns the number of variables extracted. The *extract_rules* value EXTR_PREFIX_INVALID was added in PHP 4.0.5. As of PHP 4.0.5, the *extract_rules* value EXTR_PREFIX_ALL now includes numeric variables as well.

More Examples

Example

Using all parameters:

```
<?php
$a = "Original";
$my_array = array("a" => "Cat", "b" => "Dog", "c" => "Horse");

extract($my_array, EXTR_PREFIX_SAME, "dup");

echo "\$a = $a; \$b = $b; \$c = $c; \$dup_a = $dup_a";
?>
```

Creating an Array from Variables

PHP compact() Function

Example

Create an array from variables and their values:

```
<?php
$firstname = "Peter";
$lastname = "Griffin";
$age = "41";

$result = compact("firstname", "lastname", "age");

print_r($result);
?>
```

Definition and Usage

The compact() function creates an array from variables and their values.

Note: Any strings that does not match variable names will be skipped.

Syntax

compact(*var1, var2...*)

Parameter Values

Parameter	Description
var1	Required. Can be a string with the variable name, or an array of variables
var2,...	Optional. Can be a string with the variable name, or an array of variables. Multiple parameters are allowed.

Technical Details

Return Value:	Returns an array with all the variables added to it
PHP Version:	4+
Change log:	As of version 7.3 this function issues an E_NOTICE level error if an unset variable is given

More Examples

Example

Using a string that does not match a variable, and an array of variable names:

```
<?php
$firstname = "Peter";
$lastname = "Griffin";
$age = "41";

$name = array("firstname", "lastname");
$result = compact($name, "location", "age");

print_r($result);
?>
```

Traversing Arrays

1.The foreach Construct

The most common way to loop over elements of an array is to use the foreach construct:

```
$addresses = array("spam@cyberpromo.net", "abuse@example.com");
foreach ($addresses as $value) {
echo "Processing {$value}\n";
}
```

Processing spam@cyberpromo.net
Processing abuse@example.com

PHP executes the body of the loop (the echo statement) once for each element of $addresses in turn, with $value set to the current element. Elements are processed by their internal order.

An alternative form of foreach gives you access to the current key:

```
$person = array('name' => "Fred", 'age' => 35, 'wife' => "Wilma");
foreach ($person as $key => $value) {
echo "Fred's {$key} is {$value}\n";
```

```
}
```

Fred's name is Fred
Fred's age is 35
Fred's wife is Wilma

In this case, the key for each element is placed in $key and the corresponding value is placed in $value.

The foreach construct does not operate on the array itself, but rather on a copy of it. You can insert or delete elements in the body of a foreach loop, safe in the knowledge that the loop won't attempt to process the deleted or inserted elements.

2.The Iterator Functions

Every PHP array keeps track of the current element you're working with; the pointer to the current element is known as the iterator. PHP has functions to set, move, and reset this iterator. The iterator functions are:

current()

Returns the element currently pointed at by the iterator.

reset()

Moves the iterator to the first element in the array and returns it.

next()

Moves the iterator to the next element in the array and returns it.

prev()

Moves the iterator to the previous element in the array and returns it.

end()

Moves the iterator to the last element in the array and returns it.

each()

Returns the key and value of the current element as an array and moves the itera tor to the next element in the array.

key()

Returns the key of the current element.

The each() function is used to loop over the elements of an array. It processes ele ments according to their internal order:

```
reset($addresses);
while (list($key, $value) = each($addresses)) {
echo "{$key} is {$value}<br />\n";
}
```

0 is spam@cyberpromo.net
1 is abuse@example.com

This approach does not make a copy of the array, as foreach does. This is useful for very large arrays when you want to conserve memory.

The iterator functions are useful when you need to consider some parts of the array separately from others. Example 5-1 shows code that builds a table, treating the first

index and value in an associative array as table column headings.
Example 5-1. Building a table with the iterator functions

```
$ages = array(
'Person' => "Age",
'Fred' => 35,
'Barney' => 30,
'Tigger' => 8,
'Pooh' => 40
);
// start table and print heading
reset($ages);
list($c1, $c2) = each($ages);
echo("<table>\n<tr><th>{$c1}</th><th>{$c2}</th></tr>\n");
// print the rest of the values
while (list($c1, $c2) = each($ages)) {
echo("<tr><td>{$c1}</td><td>{$c2}</td></tr>\n");
}
// end the table
echo("</table>");
```

3.Using a for Loop

If you know that you are dealing with an indexed array, where the keys are consecu tive integers beginning at 0, you can use a for loop to count through the indices. The for loop operates on the array itself, not on a copy of the array, and processes ele ments in key order regardless of their internal order.
Here's how to print an array using for:

```
$addresses = array("spam@cyberpromo.net", "abuse@example.com");
$addressCount = count($addresses);
for ($i = 0; $i < $addressCount; $i++) {
$value = $addresses[$i];
echo "{$value}\n";
}
```

spam@cyberpromo.net
abuse@example.com

4.Calling a Function for Each Array Element

PHP provides a mechanism, array_walk(), for calling a user-defined function once per element in an array:

```
array_walk(array, callable);
```

The function you define takes in two or, optionally, three arguments: the first is the element's value, the second is the element's key, and the third is a value supplied to array_walk() when it is called. For instance, here's another way to print table col

umns made of the values from an array:

```
$printRow = function ($value, $key)
{
print("<tr><td>{$key}</td><td>{$value}</td></tr>\n");
};
$person = array('name' => "Fred", 'age' => 35, 'wife' => "Wilma");
echo "<table border=1>";
array_walk($person, $printRow);
echo "</table>";
```

A variation of this example specifies a background color using the optional third argument to array_walk(). This parameter gives us the flexibility we need to print many tables, with many background colors:

```
function printRow($value, $key, $color)
{
echo "<tr>\n<td bgcolor=\"{$color}\">{$value}</td>";
echo "<td bgcolor=\"{$color}\">{$key}</td>\n</tr>\n";
}
$person = array('name' => "Fred", 'age' => 35, 'wife' => "Wilma");
echo "<table border=\"1\">";
array_walk($person, "printRow", "lightblue");
echo "</table>";
```

If you have multiple options you want to pass into the called function, simply pass an array in as a third parameter:

```
$extraData = array('border' => 2, 'color' => "red");
$baseArray = array("Ford", "Chrysler", "Volkswagen", "Honda", "Toyota");
array_walk($baseArray, "walkFunction", $extraData);
function walkFunction($item, $index, $data)
{
echo "{$item} <- item, then border: {$data['border']}";
echo " color->{$data['color']}<br />" ;
}
```

Ford <- item, then border: 2 color->red
Crysler <- item, then border: 2 color->red
VW <- item, then border: 2 color->red
Honda <- item, then border: 2 color->red
Toyota <- item, then border: 2 color->red

The array_walk() function processes elements in their internal order.

5.Reducing an Array

A cousin of array_walk(), array_reduce() applies a function to each element of the array in turn, to build a single value:

```
$result = array_reduce(array, callable [, default ]);
```

The function takes two arguments: the running total, and the current value being processed. It should return the new running total. For instance, to add up the squares of the values of an array, use:

```
$addItUp = function ($runningTotal, $currentValue)
{
$runningTotal += $currentValue * $currentValue;
return $runningTotal;
};
$numbers = array(2, 3, 5, 7);
$total = array_reduce($numbers, $addItUp);
echo $total;
87
```

The array_reduce() line makes these function calls:

```
addItUp(0, 2);
addItUp(4, 3);
addItUp(13, 5);
addItUp(38, 7);
```

The default argument, if provided, is a seed value. For instance, if we change the call to array_reduce() in the previous example to:

```
$total = array_reduce($numbers, "addItUp", 11);
```

The resulting function calls are:

```
addItUp(11, 2);
addItUp(15, 3);
addItUp(24, 5);
addItUp(49, 7);
```

If the array is empty, array_reduce() returns the default value. If no default value is given and the array is empty, array_reduce() returns NULL.

6.Searching for Values

The in_array() function returns true or false, depending on whether the first argument is an element in the array given as the second argument:

```
if (in_array(to_find, array [, strict])) { ... }
```

If the optional third argument is true, the types of to_find and the value in the array must match. The default is to not check the data types.

Here's a simple example:

```
$addresses = array("spam@cyberpromo.net", "abuse@example.com",
"root@example.com");
$gotSpam = in_array("spam@cyberpromo.net", $addresses); // $gotSpam is true
$gotMilk = in_array("milk@tucows.com", $addresses); // $gotMilk is false
```

PHP automatically indexes the values in arrays, so in_array() is generally much

faster than a loop checking every value in the array to find the one you want. Example 5-2 checks whether the user has entered information in all the required fields in a form.

Example 5-2. Searching an array

```
<?php
function hasRequired($array, $requiredFields) {
$array =
$keys = array_keys ( $array );
foreach ( $requiredFields as $fieldName ) {
if (! in_array ( $fieldName, $keys )) {
return false;
}
}
return true;
}
if ($_POST ['submitted']) {
$testArray = array_filter($_POST);
echo "<p>You ";
echo hasRequired ( $testArray, array (
'name',
'email_address'
) ) ? "did" : "did not";
echo " have all the required fields.</p>";
}
?>
<form action="<?php echo $_SERVER['PHP_SELF']; ?>" method="POST">
<p>
 Name: <input type="text" name="name" /><br /> Email address: <input
type="text" name="email_address" /><br /> Age (optional): <input
type="text" name="age" />
</p>
<p align="center">
<input type="submit" value="submit" name="submitted" />
</p>
</form>
```

A variation on in_array() is the array_search() function. While in_array() returns true if the value is found, array_search() returns the key of the element, if found:

```
$person = array('name' => "Fred", 'age' => 35, 'wife' => "Wilma");
$k = array_search("Wilma", $person);
```

```
echo("Fred's {$k} is Wilma\n");
```

Fred's wife is Wilma

The array_search() function also takes the optional third strict argument, which requires that the types of the value being searched for and the value in the array match.

END